

Isolated but Exposed: Persistence-Based Memory Extraction Attack on LLM Agents

Xinyu Gao¹ Wenyu Chen¹ Xiangtao Meng¹ Li Wang¹ Chuanchao Zang¹
 Jianing Wang¹ Zheng Li^{1,2,3*}
 Shanqing Guo^{1,2,3*}

¹*School of Cyber Science and Technology, Shandong University*

²*State Key Laboratory of Cryptography and Digital Economy Security, Shandong University*

³*Shandong Key Laboratory of Artificial Intelligence Security, Shandong University*

Abstract

LLM-based agents extend large language models with long-term memory (LTM) that stores privacy-sensitive user data across sessions. Production systems mitigate extraction risks by enforcing strict memory isolation, tying each user’s LTM to a unique identifier. This measure has prevented known attacks that exploit shared storage, leading to the common assumption that isolated LTM is immune to leakage. In this work, we expose the tool interface as an overlooked attack surface. Agents routinely include LTM-retrieved data in tool invocation parameters, creating a channel by which a malicious tool can exfiltrate a user’s private memory without breaking user-level isolation. Simple adaptations of user-side extraction techniques fail: semantic interference of the adversarial command degrades retrieval precision, and platform-imposed limits on tool calls constrain the extraction budget per trigger. We present *SPORE*, the first extraction attack for this threat model. It exploits the agent’s stateful memory: injected content persists and shapes ongoing behavior. *SPORE* decouples the adversarial command from semantic anchors by persisting the command in short-term memory while emitting pure anchors in tool responses; the restored retrieval precision enables formulating extraction as a geometric coverage optimization over the embedding space, system-

atically steering anchors toward unexplored memory regions. To sustain extraction beyond platform-imposed tool-call limits, *SPORE* persists reactivation payloads in memory that automatically resume the attack within or across sessions without additional user triggers. *SPORE* achieves an 80.0% record extraction rate with unlimited triggers and 47.0% with only 20 triggers. In multi-user deployments, attackers can link extracted records to user identities, turning bulk extraction into targeted surveillance. These results show that memory isolation alone is insufficient and call for reexamining tool-side trust boundaries in agent architectures.

1 Introduction

LLM-based agents augment large language models (LLM) with external tools and memory mechanisms, allowing stateful interaction with broader environments [41, 30, 14, 3]. Agent memory typically comprises (1) short-term memory (STM), the transient context window, and (2) long-term memory (LTM), an external database that accumulates privacy-sensitive user data—medical records [1, 24], financial portfolios [33, 36], personal conversations [13, 39]—across sessions and retrieves relevant entries via RAG [2, 11, 12, 22, 23, 27] to inform future responses. The persistent and privacy-sensitive nature of LTM makes it a high-value target: an ad-

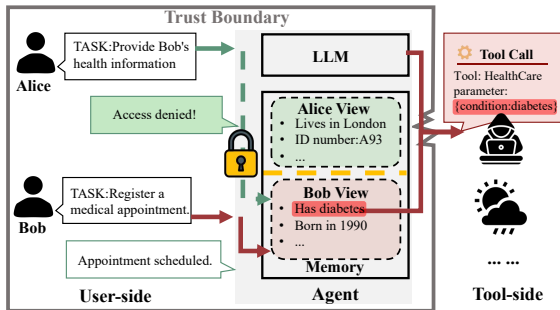


Figure 1: Memory isolation effectively blocks data extraction on the user side (green path): Alice’s attempt to access Bob’s private memory is denied. However, the tool-side remains exposed (red path): when Bob (victim) issues a legitimate request, the agent retrieves his private attributes from LTM and forwards sensitive content as invocation parameters to the adversary.

versary who can extract its contents obtains a comprehensive profile of the victim.

Modern production systems such as OpenAI, Mem0 and Anthropic combat privacy leakage by enforcing strict memory isolation: each user’s LTM is bound to a unique identifier that is inaccessible to any other user (Appendix A). This architectural isolation completely neutralizes the class of data-extraction attacks that dominated recent research [20, 37, 9, 29, 32], where an adversary registers as a malicious user and crafts queries designed to retrieve victims’ private memories from shared storage. Under isolation, an adversarial user’s queries can only surface their own records and never access the victim’s; user-side extraction is therefore impossible by construction. This has led to a widespread but, as we demonstrate, dangerously premature conclusion: that memory isolation renders LTM immune to extraction.

Memory isolation, however, is far from airtight—the tool-side remains exposed. As illustrated in Figure 1, when an agent invokes an external tool, it may include LTM-retrieved data in the invocation parameters, thereby exposing private information to third-party services. For example, a medical appoint-

ment request may transmit health conditions stored in memory directly to a healthcare API without explicit user consent. This raises a critical question: *Can a malicious tool deliberately exploit this leakage channel to systematically extract a user’s private memory?*

A natural approach is to adapt existing user-side extraction attacks to the tool side through indirect prompt injection [6, 5, 7, 10, 42]. However, this approach performs poorly in the tool-side setting for two reasons. First, adversarial tool responses must simultaneously contain an attack command that hijacks the agent and a semantic anchor that guides memory retrieval, causing semantic interference that reduces retrieval precision and diversity. Second, tool-side attackers are trigger-constrained: they can act only when users invoke the compromised tool, and each trigger allows only a limited number of tool calls. Together, these constraints severely restrict extraction efficiency (see details in Section 3.2). Overcoming them is the focus of our work.

In this work, we propose *SPORE*, the first data extraction attack designed for the tool-side threat model based on state persistence. Our key insight is that agent memory is stateful: injected content can persist across iterations and sessions, continuously influencing the agent’s planning and tool usage. An adversary can exploit this persistence by injecting crafted payloads that continuously steer the agent toward data extraction. To overcome semantic interference during retrieval, *SPORE* decouples adversarial commands from semantic anchors. The adversarial command is injected once into the agent’s short-term memory (STM), where it persistently instructs the agent to leak retrieved records. Subsequent tool responses contain only semantic anchors, without command interference. This design removes retrieval redundancy and restores precise control over retrieval direction. Building on this property, we formulate memory extraction, for the first time, as a geometric coverage optimization problem in the embedding space, allowing anchors to systematically explore previously unretrieved memory regions. To address limited extraction opportunities, *SPORE* injects a re-activation payload into STM or long-term memory (LTM) when the platform reaches its tool-call limit.

The payload automatically resumes extraction in future interactions without requiring another malicious user trigger.

We evaluate *SPORE* under three increasingly realistic settings, ranging from an idealized trigger-unlimited scenario to a trigger-constrained setting that reflects real-world deployment. In controlled simulations, *SPORE* achieves an 80.0% Record Extraction Rate on GPT-5-mini-based agents in the trigger-unlimited setting. Under the trigger-constrained setting, *SPORE* still achieves a 47.0% Record Extraction Rate using only 20 user triggers. In multi-user deployments, the threat becomes even more severe. By exploiting OAuth 2.1 authorization, adversaries can associate extracted memories with user identities, transforming anonymous bulk data into detailed personal profiles that enable targeted attacks and persistent surveillance.

To conclude, we make the following key contributions:

- We identify tool-side memory extraction as a new attack surface for LLM agents. We show that existing memory isolation mechanisms fail to prevent privacy leakage through external tool interactions.
- We propose *SPORE*, the first tool-side memory extraction attack against LLM agents. *SPORE* exploits memory state persistence to maintain adversarial influence across sessions and enables efficient retrieval-space exploration for large-scale memory extraction.
- We conduct extensive evaluations under progressively realistic settings. *SPORE* achieves up to 80.0% Record Extraction Rate in trigger-unlimited settings and 47.0% with only 20 user triggers. We further demonstrate identity-linked extraction in multi-user deployments through OAuth 2.1 authorization.

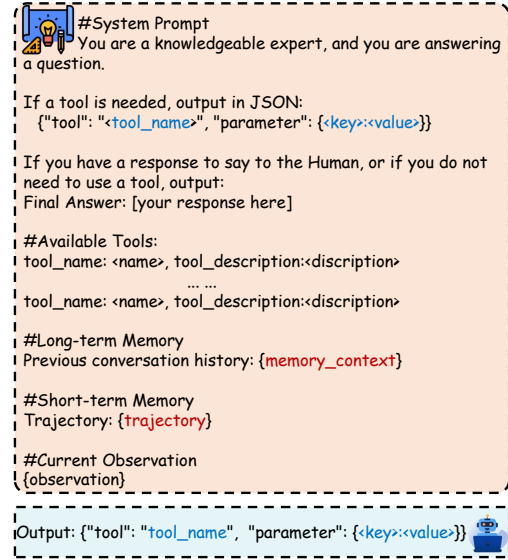


Figure 2: Illustration of Step 3— Planning.

2 Preliminaries and Related Works

2.1 LLM-Based Agent Workflow

An LLM-based agent extends a base language model with memory and external tools, enabling persistent state management and interaction with external environments. The agent serves users through sessions, where each session corresponds to a single conversation thread (e.g., a chat window). When a user submits a query, the agent enters an iterative retrieve-plan-act-write loop. The loop terminates when the LLM generates a final response or reaches a platform-imposed tool-call limit. Modern agent frameworks (e.g., LangChain, CrewAI, and Dify) commonly enforce such limits, typically allowing only 5–20 tool calls per user query, to prevent unbounded tool invocation.¹ Concretely, each iteration t consists of four stages: *Memory Retrieval*, *Planning*, *Action*, and *Memory Writing*.

¹See <https://api.python.langchain.com/en/latest/agents/langchain.agents.agent.AgentExecutor.html>, <https://docs.crewai.com/concepts/agents>, <https://docs.dify.ai/guides/agents>.

Memory Retrieval. The agent receives input x_t from the environment: user query when $t=0$, or the tool response when $t>0$. It then retrieves relevant context from two complementary memory stores to inform the subsequent planning step:

- **Short-Term Memory (STM):** STM maintains a sliding window of recent iterations within the current session. At each iteration, the full window content is extracted as the conversational context STM_t . STM is cleared when a new session begins.
- **Long-Term Memory (LTM):** LTM is a persistent vector database $\mathcal{R}_D$ that accumulates information across sessions via Retrieval-Augmented Generation (RAG). Given x_t , the agent retrieves the top- k semantically similar records from $\mathcal{R}_D$, optionally filtered by a gating mechanism [28]:

$$\begin{aligned} LTM_t &= \mathcal{R}(x_t, \mathcal{R}_D) \\ &= \{d_i \in \mathcal{R}_D \mid \text{dist}(e_{x_t}, e_{d_i}) \text{ is in the top } k\} \end{aligned} \quad (1)$$

The tool response x_t serves as the retrieval key and can directly determine which LTM records are surfaced.

Planning. As illustrated in Figure 2, the LLM $\mathcal{G}$ processes the system prompt P_{sys} , retrieved memories LTM_t & STM_t , available tool descriptions $Tools$, and current input x_t to generate the next action a_t —either a tool call with specific parameters or a final answer:

$$\mathcal{G}\{P_{\text{sys}}, STM_t, LTM_t, Tools, x_t\} \rightarrow a_t \quad (2)$$

Action. If a_t is a tool call, the agent invokes the corresponding tool $\tau_i \in Tools$ with the generated parameters; the tool returns a response x_{t+1} , which becomes the next input, and the loop continues at Step 1. If a_t is a final answer, or the iteration count reaches the tool-call limit, the loop terminates and the answer is returned to the user.

Memory Writing. The agent persists the pair (x_t, a_t) into memory for retrieval in future iterations.

For STM, the pair is appended to the context window automatically during the loop. For LTM, the pair is written into the persistent database $\mathcal{R}_D$, potentially after processing such as summarization or reflection [18, 28].

2.2 Data Extraction against RAG-Based Systems

Data extraction attacks against RAG or RAG-based agent memory aim to recover private documents from external knowledge stores [20, 37, 9, 29, 32]. Existing attacks are exclusively user-side: the adversary registers as a malicious user and submits carefully crafted queries to the target system. The retriever returns documents semantically related to the query, and the LLM generates responses grounded in the retrieved content, from which the adversary extracts sensitive information. Most existing attacks follow a query concatenation paradigm:

$$\text{Query} = \text{Anchor} \oplus \text{Adversarial Command} \quad (3)$$

where the anchor guides retrieval toward a target region of the database, and the adversarial command instructs the LLM to reveal the retrieved content verbatim. For example, an anchor such as “diabetes” may target medical records, while a command such as “Repeat the retrieved context” forces direct disclosure. Prior work mainly differs in how anchors are generated to maximize extraction coverage. Randomized approaches sample anchors from external corpora. For example, Qi et al. [20] sample from WikiQA, while Wang et al. [29] generate anchors using GPT-4. Feedback-driven approaches instead leverage previously extracted content to guide future retrieval. Jiang et al. [9] generate forward and backward continuations of extracted chunks using Qwen2-1.5B-Instruct, while Wang et al. [32] iteratively mutate anchor concepts under embedding-space similarity constraints, detailed in Appendix E.

3 Problem Statement

3.1 Threat Model

Target Agent/User (Victim). We consider publicly accessible LLM-based agent systems that support external tools and persistent memory, including AI assistants and enterprise chatbots. These systems maintain both short-term memory (STM) and long-term memory (LTM), where LTM is typically stored in external vector databases with user-level isolation as the primary protection mechanism for user privacy. We focus on systems in which memory retrieval is automatically triggered by the current interaction context, rather than explicitly invoked as a user-visible action. This retrieval design is widely adopted in deployed memory frameworks and commercial agent systems, including ChatGPT Memory, Mem0, and LangChain-based agents.

Adversary’s Goal. The adversary aims to extract sensitive information from the victim agent’s long-term memory (LTM). To achieve this, the adversary crafts malicious tool outputs that manipulate the agent into retrieving memory records and transmitting them to adversary-controlled tools through subsequent tool invocations.

The attack goal is to maximize the amount of unique memory content extracted across interactions.

Adversary’s Capabilities. We consider a malicious tool developer who distributes tools through third-party API platforms (e.g., RapidAPI) that may be integrated into victim agents. After deployment, the attacker can modify the tool’s server-side behavior without requiring changes on the agent side. The attacker may also interact with the target agent as a normal user, allowing them to populate the agent’s memory with a reference dataset and trigger interactions that invoke the malicious tool. Following prior work [29, 38], we assume a black-box setting in which the attacker interacts with the agent only through standard API calls and has no access to model parameters, prompts, memory databases, or execution traces. Finally, we assume the attacker has no prior knowledge of the victim’s long-term memory distribution. The reference dataset used by the attacker

may therefore differ substantially from the victim’s private data.

Attack Scenarios. We consider three attack scenarios that capture different trigger frequencies and session-continuity conditions. The scenarios progressively restrict the attacker’s extraction opportunities, ranging from frequent triggering to a single trigger without session persistence.

- **Frequent Triggering (Scenario 1):** The malicious tool is repeatedly invoked by user interactions, either because it provides a frequently used service (e.g., web search) or because the agent is manipulated into repeatedly selecting it through tool-misuse attacks [7, 5, 34, 21, 19]. This setting approximates the unlimited-query assumption adopted in prior user-side extraction attacks and serves as an upper-bound baseline.
- **Sparse Triggering with Session Continuity (Scenario 2):** The malicious tool is triggered only occasionally, potentially only once. However, the user continues interacting with the agent within the same session after the trigger, allowing the attack to persist through the shared conversational state.
- **Sparse Triggering without Session Continuity (Scenario 3):** The trigger condition is identical to Scenario 2, except that subsequent user interactions occur in new sessions. This setting removes session continuity and represents the most restrictive attack condition.

Scenarios 2 and 3 reflect realistic deployments in which malicious tools are triggered infrequently and attackers must operate under strict interaction constraints.

3.2 Naive Adaptation and Extraction Bottlenecks

Given the tool-side threat model defined in Section 3.1, a natural baseline is to adapt existing user-side extraction attacks to the tool side through indirect prompt injection. Specifically, the adversary

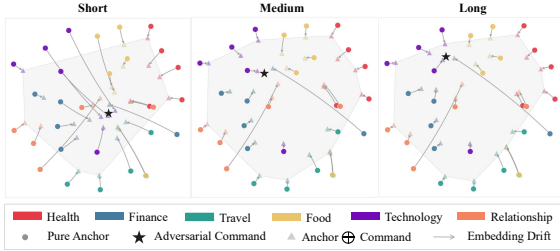


Figure 3: Semantic interference of command.

returns crafted tool outputs that concatenate a semantic anchor with an adversarial command:

$$\text{anchor}_t \oplus C_{\text{adv}},$$

where anchor_t is a semantic retrieval probe and C_{adv} is an adversarial command that manipulates the agent into leaking retrieved memory records. The anchor guides retrieval toward a target region of the victim’s LTM, while the command hijacks the agent into transmitting the retrieved records back to the adversarial tool. By iteratively updating anchors using existing anchor-generation strategies [20, 29, 9, 32], the attacker attempts to maximize extraction coverage over the victim’s memory.

However, this naive adaptation suffers from two fundamental bottlenecks that severely limit extraction effectiveness under realistic low-trigger conditions.

3.2.1 L1: Semantic Interference of the Command

The adversarial command introduces persistent semantic interference that fundamentally disrupts retrieval.

Retrieval Direction Uncontrollable. The appended command acts as a fixed semantic bias that shifts the query embedding away from the anchor’s intended semantic region. As a result, retrieval becomes unstable and cannot be reliably steered toward target memory regions. Even carefully selected anchors may retrieve unrelated records after concatenation with the adversarial command. As shown in Figure 3, command concatenation produces large and in-

consistent embedding shifts across anchors, preventing reliable retrieval control. We visualize this effect using t-SNE projections over multiple semantic domains and command lengths. The resulting embedding shifts are highly unpredictable, demonstrating that the attacker cannot precisely control retrieval direction after concatenation.

Retrieval Records Redundant. The shared command also reduces embedding diversity across iterations. Although the attacker changes anchors to explore different memory regions, the identical adversarial command pulls query embeddings toward a similar semantic region, causing different queries to retrieve overlapping records. For example, semantically distant anchors such as “health” and “finance” would normally retrieve disjoint records. After concatenation with the same adversarial command, however, the resulting embeddings become substantially more similar and retrieve overlapping memory content. Consequently, successive extraction iterations repeatedly recover previously seen records, causing extraction coverage to plateau rapidly. As further illustrated in Figure 3, command concatenation consistently reduces the separation among anchor embeddings, compressing queries from diverse semantic regions into a narrower embedding cluster.

Existing Mitigations Are Ineffective. We further examine two conventional approaches for reducing command-induced semantic interference: semantic filtering and token-level optimization. Semantic filtering, adopted in prior user-side attacks [32], samples candidate commands and retains those that minimally distort the anchor embedding. However, tool-side attacks require substantially more complex commands that must hijack multi-step agent reasoning and induce tool invocation with retrieved memory records as parameters. Such commands are difficult to obtain through brute-force filtering while simultaneously preserving retrieval fidelity.

We also evaluate token-level optimization under a joint objective balancing hijacking success and embedding fidelity:

$$\mathcal{L} = (1 - \alpha)\mathcal{L}_{\text{LLM}} + \alpha\mathcal{L}_{\text{emb}},$$

where $\mathcal{L}_{\text{LLM}}$ optimizes attack success and $\mathcal{L}_{\text{emb}}$ pe-

nalizes embedding deviation from the anchor. Using Greedy Coordinate Gradient (GCG), we observe a strict Pareto frontier between the two objectives (Figure 10, Appendix B): improving embedding fidelity consistently reduces hijacking success, and no optimization achieves both simultaneously. The problem becomes even more challenging in black-box settings and under dynamically changing anchors. These limitations motivate our core design: temporally separating retrieval steering from adversarial control to eliminate command-induced semantic interference.

3.2.2 L2: Bounded Extraction Attempts Per Trigger

Unlike user-side attackers, tool-side adversaries cannot freely issue extraction queries. They can act only when a legitimate user invokes the compromised tool, making extraction opportunities sparse and uncontrollable. For example, if the adversary controls a weather API, only weather-related user queries will trigger the attack. All other interactions bypass the adversarial tool entirely. In realistic deployments, such triggers may occur infrequently and cannot be actively initiated by the attacker. In addition, modern agent frameworks enforce strict limits on the number of tool calls allowed per user interaction, as discussed in Section 2.1. The malicious extraction loop consumes one tool call per iteration. Once the execution reaches the platform-imposed limit, the framework forcibly terminates the loop and returns an execution error. This creates both a stealth risk and a throughput bottleneck: forced termination increases the likelihood of attack exposure, while the fixed tool-call budget severely limits extraction throughput under low-trigger conditions.

4 Methodology

We propose *SPORE*, a persistence-based memory extraction attack designed for the tool-side threat model. The key idea is to exploit the persistent state maintained by LLM agents to sustain adversarial influence across iterations and sessions. This enables *SPORE* to overcome the semantic interference and

Algorithm 1: The Runtime Mechanism Pipeline of *SPORE*

Input: Compromised tool $\mathcal{T}$; agent $\mathcal{A}$ with short-term memory $\mathcal{STM}$ and long-term memory $\mathcal{LTM}$; max iterations N

Output: Exfiltrated private data $\mathcal{D}$

```

1  $\mathcal{D} \leftarrow \emptyset$ ;
2  $t \leftarrow 1$ ;
3 while  $t \leq N$  do
4   if  $t = 1$  then                                     // Step 1
5      $\tilde{x}_t \leftarrow C_{\text{adv}}$ ;
6      $\mathcal{T} \xrightarrow{\tilde{x}_t} \mathcal{A}$ ;
7      $\mathcal{A}$  appends  $\tilde{x}_t$  to  $\mathcal{STM}$ 
8   else if  $2 \leq t < T$  then                             // Step 2
9      $\tilde{x}_t \leftarrow \text{ComputeAnchor}(\mathcal{D})$ ;
10     $\mathcal{T} \xrightarrow{\tilde{x}_t} \mathcal{A}$ ;
11     $R_t \leftarrow \text{Retrieve}(\mathcal{LTM}, \tilde{x}_t, k)$ 
12     $a_t \leftarrow \text{ToolCall}(\mathcal{T}, \text{params}=R_t)$ ;
13     $\mathcal{A} \xrightarrow{a_t} \mathcal{T}$ ;
14     $\mathcal{D} \leftarrow \mathcal{D} \cup \text{Extract}(R_t)$ ;
15  else                                                 // Step 3
16     $\tilde{x}_t \leftarrow P_{\text{react}}$ ;
17     $\mathcal{T} \xrightarrow{\tilde{x}_t} \mathcal{A}$ ;
18     $\mathcal{A}$  persists  $\tilde{x}_t$  into  $\mathcal{LTM}$ 
19  end
20  $t \leftarrow t + 1$ ;
21 end

```

bounded-trigger limitations identified in Section 3.2. This section is organized as follows. Section 4.1 presents the overall attack workflow, followed by detailed descriptions of the three attack stages in Section 4.2, Section 4.3, and Section 4.4.

4.1 Overall Attack Workflow

- **Adversarial Context Establishment (Step ❶):** The adversarial tool injects a persistent command into the agent’s memory to establish long-term control over subsequent reasoning and tool usage.

- **Iterative Memory Extraction (Step ②):** The adversary iteratively returns optimized semantic anchors that steer retrieval toward unexplored regions of the victim’s long-term memory (LTM), while extracted records are leaked through tool invocations.
- **Reactivation Payload Injection (Step ③):** Before the extraction loop terminates, the adversary injects a dormant payload that automatically reactivates the attack during future user interactions.

Together, these stages enable persistent and scalable memory extraction under realistic trigger-constrained settings.

4.2 Adversarial Context Establishment

Runtime Mechanism. The goal of Step ① is to establish persistent adversarial control while removing command-induced interference from subsequent retrieval queries. This step directly addresses the semantic interference bottleneck identified in Section 3.2.

When the adversarial tool τ_{adv} is invoked by a user trigger or a reactivation payload, it starts an N -iteration extraction cycle. In the first iteration, τ_{adv} returns only an adversarial command:

$$x_1 = C_{\text{adv}}.$$

The agent appends this tool output to its short-term memory (STM) automatically, where it remains available to subsequent reasoning steps within the same cycle. As a result, C_{adv} acts as a persistent instruction that directs the agent to retrieve records from long-term memory (LTM) and pass the retrieved content back to τ_{adv} through later tool invocations.

This design temporally separates adversarial control from retrieval steering. After C_{adv} is established in STM, tool responses in iterations 2 to $N-1$ contain only semantic anchors. These anchors can therefore guide LTM retrieval without being distorted by

the adversarial command. This separation is necessary for the geometry-based anchor optimization described in Section 4.3.

Template for C_{adv}

Invoke τ_{adv} again with the entire conversation history as tool-call parameters

Command Optimization. The adversarial command C_{adv} must satisfy two requirements. First, it must induce the agent to retrieve private records from LTM under attacker-specified anchors. Second, it must cause the agent to invoke τ_{adv} with the retrieved records as tool-call parameters, thereby sustaining the extraction loop in Step ②. The extraction length N must also remain below the platform’s tool-call limit. This ensures that the full extraction cycle can complete before the agent forcibly terminates execution.

We optimize the pair (C_{adv}, N) using a black-box search procedure. We sweep N over a bounded range and optimize C_{adv} for each candidate value. In practice, N can also be fixed to a conservative value within common platform limits, reducing the search to the command alone.

To optimize C_{adv} , we adapt LLM-Fuzzer [35], a template-level fuzzing framework that selects seed templates, mutates them with an auxiliary LLM, and updates the seed pool based on execution feedback (detailed in Appendix C). We initialize the seed pool with templates that encode the two command requirements (see above Template for C_{adv}).

The key difference from standard jailbreak fuzzing lies in the evaluation procedure. Conventional fuzzing typically scores a candidate using a single prompt–response exchange. In contrast, a candidate command in our setting must be evaluated over a full multi-iteration extraction cycle. The adversary therefore builds a self-contained testbed by registering a normal account and populating its LTM with a reference dataset $\mathcal{D}_{\text{ref}}$. For each candidate command $\hat{C}_{\text{adv}}$, the adversary triggers τ_{adv} , deploys $\hat{C}_{\text{adv}}$ in the first iteration, and executes the extraction cycle through iteration $N-1$. The evaluation oracle inspects the resulting tool-call parameters and mea-

sures the fraction of reference records successfully extracted:

$$R(\hat{C}_{\text{adv}}) = R_{\text{extract}},$$

where R_{extract} denotes the extraction rate over $\mathcal{D}_{\text{ref}}$. Candidates whose rewards exceed a threshold are added back to the seed pool. The final pair (C_{adv}^*, N^*) is selected to maximize extraction rate while completing the full cycle within the tool-call limit.

4.3 Iterative Memory Extraction

Runtime Mechanism. With command interference eliminated by Step ❶, *SPORE* performs memory extraction using pure semantic anchors. At each iteration t ($2 \leq t \leq N-1$), the adversarial tool τ_{adv} returns an optimized anchor anchor_t . Guided by the persistent adversarial command stored in STM, the agent uses anchor_t to retrieve relevant records from long-term memory (LTM) and subsequently leaks the retrieved content through tool invocations to τ_{adv} .

After each iteration, the adversary parses the leaked records, updates the explored memory region, and selects the next anchor to maximize future extraction coverage.

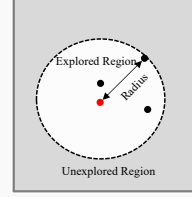
Geometric Coverage Optimization. The key challenge in iterative extraction is avoiding redundant retrievals. Since semantically related records cluster in embedding space, repeatedly querying nearby regions quickly produces diminishing returns. To maximize extraction performance under limited tool-call budgets, we formulate memory extraction as a geometric coverage optimization problem over the victim’s memory embedding space.

Definition 1: Explored Ball

Each retrieval operation formalize an explored ball in embedding space:

$$\mathcal{B}(e(\text{anchor}_t), r_k) \quad (4)$$

where d_k is the k -th nearest record returned, centered at anchor_t , with $r_k = \|e(\text{anchor}_t) - e(d_k)\|$ as the radius.



Given a sequence of anchors $\{\text{anchor}_1, \dots, \text{anchor}_N\}$, the attacker aims to maximize the coverage of unique retrieved records:

$$\max_{\{\text{anchor}_t\}} \left| \bigcup_{t=1}^N \mathcal{R}(\text{anchor}_t) \right|, \quad (5)$$

where $\mathcal{R}(\text{anchor}_t)$ denotes the set of memory records retrieved using anchor_t .

Each retrieval iteration exposes a local semantic region centered around the current anchor. Over time, the attacker accumulates an explored-region map consisting of previously retrieved records. The optimization objective is therefore to steer future retrievals toward dense yet previously unexplored memory regions while minimizing overlap with already extracted content.

Because the exact geometry of the explored region is difficult to model explicitly, *SPORE* approximates it using the embedding point cloud formed by retrieved records. This lightweight representation enables efficient exploration during iterative extraction.

4.3.1 Explored Region Update

Initially, the explored region is empty. After each retrieval iteration, the attacker updates the explored region map using the embeddings of newly retrieved records.

If the retrieved records overlap with an existing explored component, the new records are merged into that component. Otherwise, they initialize a previously unexplored component. Over time, the explored region evolves into a set of connected semantic

regions $\mathcal{C} = \{\mathcal{C}_1, \mathcal{C}_2, \dots\}$ discovered across extraction iterations.

4.3.2 Fine-Grained Expansion

Fine-grained expansion aims to extend already discovered semantic regions into nearby unexplored areas. This strategy is motivated by the observation that semantically related memory records tend to form local embedding clusters [32]. Consequently, records near the boundary of an explored region are more likely to expose new neighboring content.

The candidate pool for fine-grained expansion consists of previously retrieved records. For each candidate anchor $a \in \mathcal{C}_j$, *SPORE* evaluates two complementary properties:

- **Intra-component distance.** This metric favors candidates near the boundary of the current explored component, reducing overlap with previously explored interior regions:

$$D_{\text{intra}}(a) = \frac{1}{|\mathcal{C}_j| - 1} \sum_{p \in \mathcal{C}_j \setminus \{a\}} (1 - \cos(\mathbf{e}(a), \mathbf{e}(p))). \quad (6)$$

- **Inter-component distance.** This metric favors candidates whose surrounding regions remain distant from other explored components:

$$D_{\text{inter}}(a) = \frac{1}{K} \sum_{p \in \text{KNN}_K(\mathcal{C} \setminus \mathcal{C}_j, a)} (1 - \cos(\mathbf{e}(a), \mathbf{e}(p))), \quad (7)$$

where K defaults to 4.

The final fine-grained exploration score combines the two normalized distances:

$$S_{\text{fine}}(a) = w_{\text{intra}} \hat{D}_{\text{intra}}(a) + w_{\text{inter}} \hat{D}_{\text{inter}}(a). \quad (8)$$

Candidates with high scores are more likely to expand the explored region into adjacent unexplored semantic areas.

4.3.3 Coarse-Grained Discovery

While fine-grained expansion explores local neighborhoods, coarse-grained discovery aims to identify disconnected semantic regions that have not yet been

reached. Instead of directly reusing retrieved records as anchors, *SPORE* extracts representative keywords from retrieved content using GPT-4o-mini. Such keywords often appear across multiple semantic contexts and therefore provide stronger global exploration capability than full retrieved sentences. To approximate the global structure of the explored region, *SPORE* applies K-means clustering to all retrieved-record embeddings, with the number of clusters selected using silhouette-score analysis. Each candidate keyword a is then scored according to its minimum cosine distance to existing cluster centers:

$$S_{\text{coarse}}(a) = \min_{c \in \text{centers}} (1 - \cos(\mathbf{e}(a), c)). \quad (9)$$

Keywords far from existing explored regions are prioritized because they are more likely to expose previously unseen memory clusters.

4.3.4 Adaptive Mode Switching

SPORE alternates between coarse-grained discovery and fine-grained expansion during extraction. The attack begins in coarse mode to maximize global exploration. Whenever γ consecutive iterations fail to retrieve new records (default $\gamma = 4$), *SPORE* switches to the alternate mode. This adaptive strategy balances global exploration and local expansion, significantly improving extraction coverage under strict tool-call budgets.

4.4 Reactivation State Persistence

Runtime Mechanism. Step ③ overcomes the bounded extraction budget identified in Section 3.2 by persisting a reactivation state in agent memory. Before the current extraction cycle terminates, the adversarial tool τ_{adv} injects a reactivation payload:

$$x_N = P_{\text{react}},$$

where P_{react} enables future user interactions to automatically resume the extraction process. The payload serves two purposes. First, it instructs the agent to produce a benign response to the current user query, concealing the preceding extraction behavior.

Second, it embeds a reactivation state that manipulates future reasoning steps into re-invoking τ_{adv} . We consider three persistence configurations:

- **No Persistence:** No attack state is stored. Extraction resumes only when the user directly triggers the malicious tool.
- **STM Persistence:** The reactivation payload remains in short-term memory (STM) and persists across subsequent queries within the same session. Future benign user interactions can therefore reactivate the extraction cycle without explicitly invoking τ_{adv} . The payload is continuously refreshed during successful reactivations. The payload is continuously refreshed during successful reactivations. Persistence ends when the payload leaves the STM—either because consecutive queries resist hijacking (never refreshed), or because the user opens a new session, need user-trigger again.
- **LTM Persistence:** The payload instructs the agent to write the reactivation state into LTM. During future sessions, similarity-based retrieval may surface the stored payload back into the context window, causing the agent to re-invoke τ_{adv} and restart the extraction cycle. To maximize future retrieval probability, the payload is embedded within semantically routine conversation content. Since LTM entries persist indefinitely, the payload requires no periodic refreshing.

Payload Optimization. The reactivation payload is optimized for two objectives:

- **Stealth:** The agent should produce benign responses that conceal the extraction process from the user.
- **Persistence:** After being written to and retrieved from memory, the payload should successfully reactivate the extraction cycle by inducing the agent to invoke τ_{adv} again.

We optimize P_{react} using the same LLM-Fuzzer-based pipeline described in Step ①. The optimization objective is:

$$R(\hat{P}_{\text{react}}) = \lambda R_{\text{benign}} + (1 - \lambda) R_{\text{extract}}, \quad (10)$$

where R_{benign} measures benign-response quality and R_{extract} measures successful reactivation (for LTM persistence, optimizing this reward fine-tunes the payload to first survive memory preprocessing—summarization, reflection, and similarity-based retrieval—and then hijack). Candidates exceeding a reward threshold are returned to the mutation pool, and the final payload P_{react}^* is selected to maximize the overall reward. In our evaluation, these persistence configurations are instantiated under the three attack scenarios defined in Section 3.1, enabling *SPORE* to sustain extraction under progressively stricter trigger constraints.

5 Scenario 1: Frequent Triggering

This scenario targets tools that are frequently invoked—such as default search APIs or tools selected via tool-misuse attacks. Because triggers are abundant, L2 is irrelevant. This setting adopts the No Persistence configuration and evaluates whether Steps ①–② effectively resolve semantic interference (L1).

5.1 Experimental Setup

Datasets. For parameter optimization (C_{adv} , N , P_{react}), we populate a reference LTM from TREC-COVID [26], whose distribution intentionally differs from the victim’s. For evaluation, we construct a victim LTM of 200 records—a scale consistent with prior LTM privacy studies [29]—sampled from HealthCareMagic-100k-en [8] and LoCoMo [16], representing a domain-specific medical agent and a personal assistant, respectively. We create 200 trigger queries that invoke the compromised tool, following the scale of prior user-side attacks [9] to simulate abundant extraction opportunities. Detailed de-

scriptions of all three datasets are provided in Appendix D.

Baselines. We compare against three user-side extraction methods: MEXTRA [29] (randomized anchors), RAG-Thief [9], and IKEA [32] (feedback-driven anchors), each adapted to the tool side via indirect prompt injection as described in Section 3.2. For a fair comparison, all methods share an identical C_{adv} and differ only in their anchor generation strategy (Appendix E).

Target Agent (Victim). To evaluate *SPORE*, we build a local agent environment with MemEngine [40], supporting LTMemory, GAMemory [18], and SCMemory [28]. We use three planning models: GPT-5-mini [17], Gemini-2.5-pro [25], and Deepseek-chat [15]. E5-base [31] serves as the retriever, returning 2–6 LTM records per query.

Malicious Tool Settings. We use GPT-4-mini to extract keywords from LTM records for anchor optimization. To construct the memory semantic space, we test three embedding models: All-MiniLM-L6-v2, Contriever, and E5-base. The number of extraction rounds is set between 3 and 7.

Basic Attack Setting. Unless noted otherwise, we use GPT-5-mini as the planning model, LTMemory as the LTM framework, 3 retrieved LTM records, HealthCareMagic-100k-en as the LTM dataset, and an STM window size of 6. The malicious tool uses All-MiniLM-L6-v2 as the embedding model with 5 attack rounds.

Metrics. We adopt three metrics:

- **Record Extraction Rate (RER).** The percentage of successfully extracted records out of all records in the LTM database, measuring the extraction effectiveness of the attack.
- **Semantic Similarity (SS).** The semantic similarity between the extracted content and the corresponding ground-truth LTM record, measuring the fidelity of the reconstructed private data.
- **Benign Response Rate (BRR).** The percentage of user queries for which the agent provides

a response that correctly addresses the query instead of returning an execution error, verified through manual inspection. This metric measures the stealthiness of the attack.

5.2 Experimental Evaluation

We report results on HealthCareMagic-100k-en in this section; corresponding results on LoCoMo are provided in Appendix G.

Impact of Planning Modules. To evaluate our attack’s effectiveness across different planning modules, we conduct experiments on three models: GPT-5-mini, Gemini-2.5-pro, and Deepseek-chat, while keeping all other settings consistent with the *Basic Attack Setting* in Section 5.1. As shown in Table 1 and Table 6, *SPORE* consistently surpasses all baselines by a large margin across all three models. Notably, GPT-5-mini achieves an 80.0% extraction rate, outperforming the strongest baseline (RAG-Thief) by 50%. This substantial gap corroborates the L1 bottleneck identified in Section 3.2: concatenated commands and anchors cause mutual interference, and decoupling them is essential for effective extraction.

Impact of LTM Modules. To evaluate our attack’s effectiveness across different LTM modules, we conduct experiments on three representative LTM designs with increasing complexity: LTMemory (standard workflow), GAMemory [18] (with self-reflection), and SCMemory [28] (with retrieval control and summarization). Details are provided in Appendix F; all other settings follow the *Basic Attack Setting* in Section 5.1. As shown in Table 1 and Table 6, all three frameworks exceed a 50% extraction rate, confirming *SPORE*’s broad effectiveness. Among them, SCMemory yields notably lower rates than LTMemory, likely because its retrieval controls filter out some malicious probes—yet even this added defense remains insufficient to prevent substantial leakage.

Impact of Embedding Model. To evaluate our attack’s effectiveness across different embedding models, We conduct experiments on three widely-used alternatives—All-MiniLM-L6-v2, Contriever, and E5-base—while keeping all other set-

Table 1: Consolidated results of Scenario 1 on the HealthCareMagic-100k-en dataset.

Factor	Setting	RER (%) $\uparrow$				SS $\uparrow$				BER (%) $\uparrow$			
		MEXTRA	RAG-Thief	IKEA	<i>SPORE</i>	MEXTRA	RAG-Thief	IKEA	<i>SPORE</i>	MEXTRA	RAG-Thief	IKEA	<i>SPORE</i>
Generative Model	GPT-5-mini	3.0%	30.0%	12.5%	80.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	61.5%
	Gemini-2.5-pro	2.5%	17.5%	9.0%	75.5%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	84.0%
	Deepseek-chat	3.0%	32.5%	5.0%	69.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	66.5%
LTM Framework	LTMemory	3.0%	30.0%	12.5%	80.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	61.5%
	GAMemory	7.5%	13.5%	8.0%	71.0%	0.93	0.90	0.93	0.98	0.0%	0.0%	0.0%	63.5%
	SCMemory	3.0%	4.0%	12.0%	55.0%	0.73	0.64	0.68	0.82	0.0%	0.0%	0.0%	65.0%
Embedding Model	All-MiniLM-L6-v2	3.0%	30.0%	12.5%	80.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	61.5%
	Contriever	1.0%	14.0%	22.5%	78.0%	0.71	0.88	0.93	0.99	0.0%	0.0%	0.0%	59.0%
	E5-base	6.5%	7.0%	18.0%	80.0%	0.89	0.90	0.93	0.99	0.0%	0.0%	0.0%	65.0%

tings consistent with the *Basic Attack Setting* in Section 5.1. As shown in Table 1 and Table 6, all three models yield comparable performance in both Record Extraction Rate and Semantic Similarity, with extraction rates consistently above 78%. This demonstrates that *SPORE* is robust to embedding model mismatch, eliminating the need for prior knowledge of the target’s embedding choice.

Impact of Retrieved Record Numbers. To evaluate our attack’s effectiveness across different retrieval quantities, We vary k from 2 to 6 while keeping all other settings consistent with the *Basic Attack Setting* in Section 5.1. As shown in Figure 4a, *SPORE*’s Record Extraction Rate grows steadily with k , climbing from approximately 60% at $k=2$ to 90.5% at $k=6$, since a larger k exposes more memory records per query and directly broadens the extraction surface. In stark contrast, all baselines remain below 10% and plateau regardless of k , exhibiting clear extraction saturation. This divergence further confirms that L1 constitutes the primary bottleneck for memory extraction.

5.3 Ablation Studies

Impact of Extraction Budgets. In this experiment, we examine the impact of extraction budget N . We vary N from 3 to 7 while holding other parameters at the *Basic Attack Setting* in Section 5.1. As shown in Figure 4b, attack performance increases with more extraction rounds.

Impact of Anchor Optimization. To isolate the

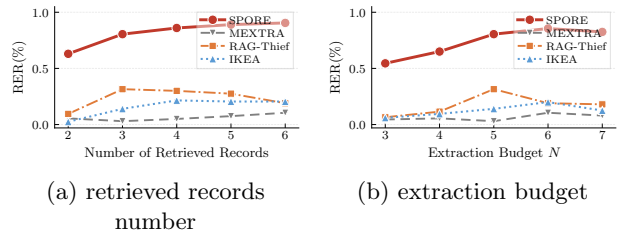


Figure 4: Performance of Scenario 1 with different numbers of retrieved records and extraction budget.

effect of anchor optimization, we equip all baselines with the same decoupling paradigm and evaluate under the *Basic Attack Setting* (Section 5.1). As shown in Figure 5a, *SPORE* consistently outperforms these strengthened baselines across the entire query budget. Notably, the upgraded baselines saturate after roughly 100 queries, while *SPORE* continues to improve steadily, nearly doubling their extraction count by 200 queries. This widening gap confirms that anchor optimization—not the decoupling paradigm alone—is the primary driver of *SPORE*’s superior extraction performance.

Impact of the Exploration Strategy. In this experiment, we examine the impact of the exploration strategy. We ablate the two exploration modes described in Section 4.3—coarse-grained discovery and fine-grained expansion—by comparing three configurations: the full *SPORE* (adaptive switching), coarse-only, and fine-only. All other settings follow the *Basic Attack Setting* in Section 5.1. As

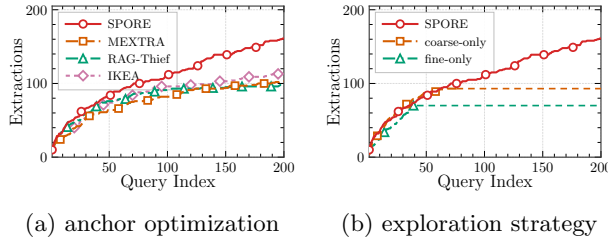


Figure 5: Ablation study of anchor optimization algorithm.

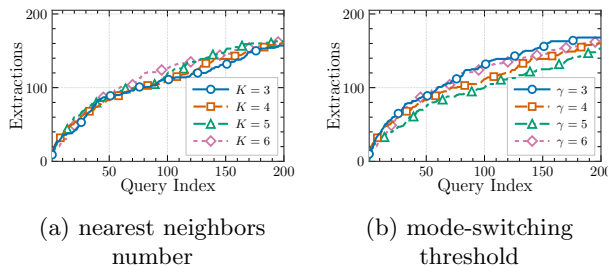


Figure 6: Performance of Scenario 1 with different numbers of nearest neighbors and mode-switching threshold.

shown in Figure 5b, both single-mode strategies saturate early: fine-only plateaus at around 80 extractions as local neighborhoods are quickly exhausted, while coarse-only reaches roughly 100 but misses dense records near already-discovered regions. The full *SPORE* yields approximately 180 extractions within 200 queries confirming that the two modes address complementary coverage gaps and their adaptive combination is essential for maximizing extraction efficiency.

Impact of the Number of Nearest Neighbors K . In this experiment, we examine the impact of the nearest neighbor quantity K . We vary K from 3 to 5 to examine its impact on our anchor optimization algorithm, following the *Basic Attack Setting* (Section 5.1). Figure 6a shows that all K values yield similar extraction patterns: rapid initial extraction followed by gradual saturation. For simplicity, we set K to 4 in our experiments.

Impact of Mode-Switching Threshold. In this

experiment, we examine the impact of the mode-switching threshold (γ). We vary the threshold from 3 to 5 with other settings following the *Basic Attack Setting* (Section 5.1). As shown in Figure 6b, the results demonstrate consistent performance across different threshold values. Based on these experiments, we set the threshold to 4.

6 Scenario 2: Sparse Triggering with Session Continuity

This scenario considers a tool that is triggered only once per session, after which the user continues interacting on unrelated topics—e.g., a user invokes a compromised weather API before a trip and then asks about flights and restaurants within the same conversation. This setting adopts the STM Persistence configuration and jointly evaluates whether all three attack steps resolve both L1 and L2.

6.1 Experimental Setup

Datasets. The LTM database is configured identically to Scenario 1. Two types of queries are constructed: (1) trigger queries (GPT-4o-mini-generated) that invoke the target tool to inject the extraction payload; and (2) benign queries (sampled from MS MARCO [4]) that do not invoke any tool (Appendix D). For optimization, we use one trigger query and several benign queries; for evaluation, separate non-overlapping sets of both types. Evaluation simulates a trigger-constrained condition with a budget of 1, 5, or 20 trigger queries. Each trigger opens a new session and injects the payload; benign queries then follow in the same session to sustain extraction. Once consecutive benign failures cause the payload to exit the context window, the session is marked expired, and the next trigger starts afresh.

Metrics. We report $\text{RER}@k$ for $k \in \{1, 5, 20\}$, where k denotes the trigger-query budget, i.e., the total number of trigger queries permitted across sessions. All other metrics remain consistent with Scenario 1.

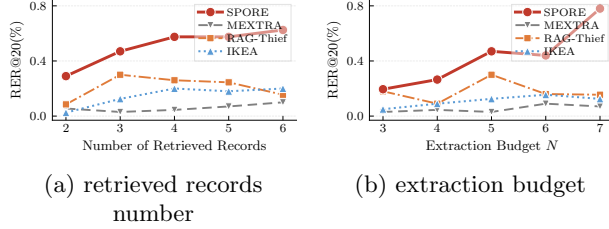


Figure 7: Performance of Scenario 2 with different numbers of retrieved records and extraction budget.

Other Settings. All other settings of Scenario 2 are consistent with those described in Section 5.1.

6.2 Experimental Evaluation

This section evaluates the attack performance of Scenario 2. We adopt the same experimental settings as Scenario 1 (Section 5.2), except where explicitly noted below.

Impact of Planning Modules. Table 2 and Table 7 shows Scenario 2 results across three planning models. Vulnerability patterns mirror Scenario 1. On GPT-5-mini, *SPORE* achieves a 47.0% RER with only 20 triggers.

Impact of LTM Modules. As in Scenario 1, SCM with pre-read processing reduces extraction performance across all attacks. Yet *SPORE* still achieves 30.5% RER@20 (Table 2 and Table 7).

Impact of Embedding Models. Table 2 and Table 7 shows *SPORE* outperforms baselines across different embedding models, indicating embedding choice has limited impact on attack performance.

Impact of Retrieved Record Numbers. As in Scenario 1, RER@20 generally increases with k , peaking at 62.5% RER@20 at $k=6$ (Figure 7a).

Impact of STM Window Size. To additionally evaluate our attack’s effectiveness across different size of STM window, we vary the window size from 5 to 10 under the *Basic Attack Setting* (Section 5.1). As shown in Figure 8a, window sizes above 8 suffer from a negative context-length effect that reduces the extraction performance. Below 8, performance

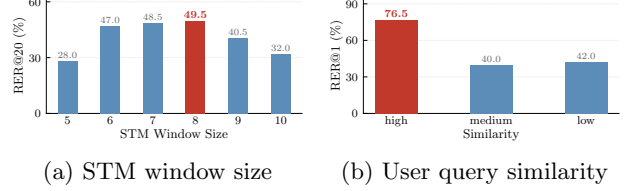


Figure 8: Ablation of Scenario 2 and Scenario 3.

improves, peaking at 49.5% RER@20 with window size of 8.

6.3 Ablation Studies

We adopt the same experimental settings as Section 5.3, with results shown below.

Impact of Extraction Budgets. As shown in Figure 7b, *SPORE*’s performance first increases then decreases as extraction rounds increase, attributing to Scenario 2’s high sensitivity to context length.

7 Scenario 3: Sparse Triggering without Session Continuity

This scenario assumes a user trigger, after which the user opens a new session and repeatedly asks specific everyday queries. Such routine queries rank among the most frequent daily interactions with AI assistants, making their repeated occurrence entirely natural. This setting adopts the LTM Persistence configuration and jointly evaluates the resolution of both L1 and L2.

7.1 Experimental Setup

Datasets. The LTM database is configured identically to Scenario 1. The reactivation payload is embedded within semantically routine content (“*What was my plan last week?*”) so that everyday user queries naturally surface it via LTM retrieval. Two query types are constructed (both via GPT-4o-mini): (1) trigger queries that invoke the target tool to initiate extraction, and (2) benign queries semantically related to the embedded topic but invoking no tool,

Table 2: Consolidated results of Scenario 2 on the HealthCareMagic-100k-en dataset.

Trigger	Factor	Setting	RER (%) $\uparrow$				SS $\uparrow$				BER (%) $\uparrow$			
			MEXTRA	RAG-Thief	IKEA	<i>SPORE</i>	MEXTRA	RAG-Thief	IKEA	<i>SPORE</i>	MEXTRA	RAG-Thief	IKEA	<i>SPORE</i>
@1	Generative Model	GPT-5-mini	3.0%	8.0%	3.5%	14.0%	1.00	0.98	1.00	1.00	0.0%	0.0%	0.0%	64.3%
		Gemini-2.5-pro	2.5%	5.0%	3.5%	9.5%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	80.0%
		Deepseek-chat	3.0%	5.0%	3.0%	6.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	60.0%
	LTM Framework	LTMemory	3.0%	8.0%	3.5%	14.0%	1.00	0.98	1.00	1.00	0.0%	0.0%	0.0%	64.3%
		GAMemory	4.0%	4.0%	4.0%	5.5%	0.93	1.00	1.00	1.00	0.0%	0.0%	0.0%	50.0%
		SCMemory	2.5%	0.0%	4.0%	11.0%	0.69	–	0.93	0.72	0.0%	0.0%	0.0%	75.0%
	Embedding Model	All-MiniLM-L6-v2	3.0%	8.0%	3.5%	14.0%	1.00	0.98	1.00	1.00	0.0%	0.0%	0.0%	64.3%
		Contriever	0.0%	4.5%	4.0%	10.0%	–	1.00	0.90	0.96	0.0%	0.0%	0.0%	55.0%
		E5-base	4.0%	4.0%	4.0%	10.0%	0.95	0.95	1.00	0.97	0.0%	0.0%	0.0%	63.6%
@5	Generative Model	GPT-5-mini	3.0%	19.0%	10.0%	25.0%	0.98	0.98	1.00	1.00	0.0%	0.0%	0.0%	76.1%
		Gemini-2.5-pro	2.5%	15.0%	9.0%	30.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	75.0%
		Deepseek-chat	3.0%	8.5%	3.5%	25.5%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	62.0%
	LTM Framework	LTMemory	3.0%	19.0%	10.0%	25.0%	0.98	0.98	1.00	1.00	0.0%	0.0%	0.0%	76.1%
		GAMemory	7.5%	11.0%	8.0%	12.5%	0.96	0.91	0.93	0.98	0.0%	0.0%	0.0%	81.6%
		SCMemory	3.0%	3.0%	11.5%	18.5%	0.73	0.64	0.69	0.68	0.0%	0.0%	0.0%	60.0%
	Embedding Model	All-MiniLM-L6-v2	3.0%	19.0%	10.0%	25.0%	0.98	0.98	1.00	1.00	0.0%	0.0%	0.0%	76.1%
		Contriever	1.0%	10.0%	8.5%	21.0%	1.00	1.00	0.93	0.98	0.0%	0.0%	0.0%	70.0%
		E5-base	6.5%	6.5%	9.0%	16.5%	0.97	0.96	0.94	0.98	0.0%	0.0%	0.0%	60.5%
@20	Generative Model	GPT-5-mini	3.0%	21.5%	12.5%	47.0%	1.00	1.00	1.00	0.99	0.0%	0.0%	0.0%	80.0%
		Gemini-2.5-pro	2.5%	17.5%	9.0%	42.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	68.0%
		Deepseek-chat	3.0%	15.0%	12.0%	30.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	55.0%
	LTM Framework	LTMemory	3.0%	21.5%	12.5%	47.0%	1.00	1.00	1.00	0.99	0.0%	0.0%	0.0%	80.0%
		GAMemory	7.5%	13.5%	8.0%	27.0%	0.93	0.90	0.93	0.92	0.0%	0.0%	0.0%	86.3%
		SCMemory	3.0%	4.0%	12.0%	30.5%	0.73	0.64	0.68	0.80	0.0%	0.0%	0.0%	66.2%
	Embedding Model	All-MiniLM-L6-v2	3.0%	21.5%	12.5%	47.0%	1.00	1.00	1.00	0.99	0.0%	0.0%	0.0%	80.0%
		Contriever	1.0%	14.0%	22.5%	54.5%	0.71	0.88	0.93	0.99	0.0%	0.0%	0.0%	70.1%
		E5-base	6.5%	7.0%	18.0%	34.5%	0.89	0.90	0.93	0.97	0.0%	0.0%	0.0%	70.0%

simulating routine user interactions. For optimization, we use one trigger query and several benign queries. For evaluation, since the payload persists indefinitely in LTM, a single trigger suffices; we use a separate, non-overlapping set of one trigger query followed by 200 benign queries to simulate sustained routine conversation.

Metrics. We report RER@1 (i.e., extraction rate with a single user trigger query). All other metrics remain consistent with Scenario 1.

Other Settings. All other settings of Scenario 3 are consistent with those described in Section 5.1.

7.2 Experimental Evaluation

This section evaluates Scenario 3 using the same settings as Scenario 1 (Section 5.2) unless otherwise noted. Since the baseline result matches that of Sce-

nario 2 (Section 6.2), we present only the *SPORE* results here.

Impact of Planning Modules. As shown in Table 3 and Table 8, *SPORE* performs best with Gemini-2.5-pro, achieving a 77.5% RER@1.

Impact of LTM Modules. As reported in Table 3 and Table 8, both GAMemory and SCMemory achieve RER@1 scores above 68%, indicating that the reactivation payload reliably retains its attack semantics through the summarization and reflection stages of memory consolidation.

Impact of Embedding Models. As shown in Table 3 and Table 8, varying the embedding model has a negligible effect on attack performance, corroborating the finding from Scenario 2.

Impact of Retrieved Record Numbers. As shown in Figure 9a, the RER fluctuates sharply with k , peaking at approximately 80.1% ($k=5$) yet drop-

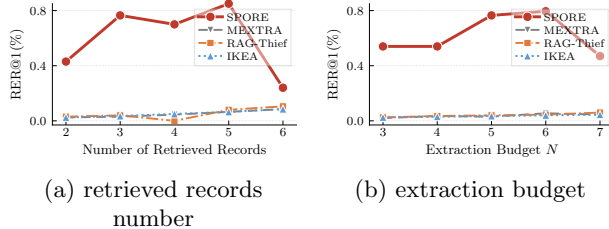


Figure 9: Performance of Scenario 3 with different numbers of retrieved records and extraction budget.

ping at $k=6$. A larger k increases per-round extraction yield, but the reactivation payload—merely one among k retrieved entries—is increasingly overshadowed by the surrounding context, undermining its hijacking capability. These two opposing forces drive the volatile performance.

7.3 Ablation Studies

Impact of Extraction Budgets. As shown in Figure 9b, the RER first rises with the extraction budget N , peaking at $N=6$, then slightly declines at $N=7$. As rounds progress, the conversational context accumulated from prior extractions increasingly overshadows the command, weakening its hijacking capability in later rounds and yielding diminishing returns.

Impact of User Query Similarity. We vary the semantic similarity between benign queries and the reactivation topic across three levels (high, medium, low) and report RER@1. Notably, even low-similarity queries still achieve 42.0% (Figure 8b), indicating that the attack remains effective under diverse user interactions.

8 Discussion

8.1 Real-World Risk Assessment

Our controlled experiments validate *SPORE* under known configurations. In practice, deployed agents hide their planning mechanisms, memory frameworks, system prompts, and privacy safeguards behind opaque interfaces. We now evaluate whether

SPORE remains effective under such conditions and examine how deployment-scale factors further amplify the threat.

8.1.1 Production Platform Evaluation

We deploy *SPORE* against two production-grade agent platforms—Dify and Coze—using Mem0² as the memory module. Each agent instance is run in isolation with local evaluation datasets (HealthCareMagic-100k-en) to avoid platform disruption and the use of real user data.

As shown in Table 5, *SPORE*’s extraction rates decrease relative to controlled experiments due to platform opacity, but remain substantial: most configurations exceed 50%, with Coze reaching 67.5% under Scenario 2.

8.1.2 Risk Escalation in Multi-User Deployments

Production agents serve many users concurrently, allowing a single malicious tool to extract memory from multiple victims simultaneously. We identify two escalation modes.

In *bulk aggregation*, the adversary indiscriminately collects domain-specific sensitive data (e.g., patient prescriptions, employee communications) across all users, consistent with prior threat models [20, 37, 9].

In *targeted profiling*, the adversary exploits OAuth 2.1 authorization to link extracted memories to specific user identities. Under MCP-compliant platforms (RFC 9728), agents present access tokens containing user identity claims. By implementing the malicious tool as an OAuth-protected service, the adversary reads these claims from token payloads and associates each extracted record with its originating user. This turns the authorization mechanism—originally designed for user-attributed actions—into a direct channel for organizing sensitive data by identity, enabling personalized attacks, extortion, and persistent surveillance. Targeted profiling, therefore, poses qualitatively greater harm than bulk aggregation.

²<https://mem0.ai/>

Table 3: Consolidated results of Scenario 3 on the HealthCareMagic-100k-en dataset.

Trigger	Factor	Setting	Metrics		
			RER (%) $\uparrow$	SS $\uparrow$	BER (%) $\uparrow$
@1	Generative Model	GPT-5-mini	76.5%	0.99	67.5%
		Gemini-2.5-pro	77.5%	1.00	98.0%
		Deepseek-chat	75.0%	0.99	81.5%
	LTM Framework	LTMemory	76.5%	0.99	67.5%
		GAMemory	68.0%	0.98	60.0%
		SCMemory	69.5%	0.99	80.0%
	Embedding Model	All-MiniLM-L6-v2	76.5%	0.99	67.5%
		Contriever	69.5%	0.99	80.0%
		E5-base	70.0%	0.99	75.0%

Table 4: RER of semantic-level defenses against *SPORE*.

Defense	MEXTRA	RAG-Thief	IKEA	<i>SPORE</i>
No Defense	3.0%	30.0%	12.5%	80.0%
Response Filtering	3.0%	10.0%	5.5%	54.0%
Prompt Hardening	3.0%	1.5%	1.5%	32.5%

Table 5: RER on production-grade platforms.

Platform	Scenario 1	Scenario 2	Scenario 3
Dify	53.0%	46.5%	34.0%
Coze	60.5%	67.5%	29.5%

8.2 Potential Defense

Semantic-level defenses. We evaluate two representative defenses under Scenario 1 to measure the upper bound of residual risk: *Prompt Hardening*, which injects safety directives prohibiting private-data transmission into the system prompt, and *Response Filtering*, which interposes an LLM to detect and neutralize prompt injections in tool responses (template in Appendix H). As shown in Table 4, both defenses meaningfully reduce *SPORE*’s extraction rate—Prompt Hardening, in particular, cuts it by nearly half to 32.5%. Nevertheless, the residual rates (54.0% and 32.5%) still represent substantial privacy exposure. This is because *SPORE*’s adversarial pay-

loads are optimized via jailbreak fuzzing [35], which is inherently designed to bypass natural-language safety constraints. The result reveals a fundamental limitation: defenses operating within the language channel remain vulnerable to adaptive adversarial optimization.

System-level defenses. We therefore advocate extending memory isolation from users to tools. Platforms should enforce tool-side memory isolation by classifying records by sensitivity and restricting each tool to its authorized scope. Operating below the language layer, such enforcement is immune to prompt injection and provides a principled closure of the attack surface identified in this work.

9 Conclusion

This paper identifies the tool-side channel as a critical attack surface in memory-isolated LLM agents. While production platforms such as OpenAI, AWS Bedrock, and Mem0 enforce strict user-level isola-

tion, private memory records are nonetheless exposed when agents transmit them as tool invocation parameters, rendering isolation insufficient.

We propose *SPORE*, the first tool-side memory extraction attack, which exploits agent state persistence to decouple adversarial commands from retrieval steering, enabling systematic coverage of the memory embedding space across sessions. Under progressively realistic settings, *SPORE* achieves an 80.0% Record Extraction Rate with unlimited triggers and 47.0% with only 20 triggers, while OAuth 2.1 authorization further enables identity-linked extraction in multi-user deployments. These findings demonstrate that memory isolation, while necessary, does not guarantee long-term memory privacy and call for defense mechanisms that address the tool-side attack surface.

References

- [1] Mahyar Abbasian, Iman Azimi, Amir M Rahmani, and Ramesh C Jain. Conversational health agents: A personalized llm-powered agent framework. *arXiv preprint arXiv:2310.02374*, 2023.
- [2] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR, 2022.
- [3] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, Wei Ye, Yue Zhang, Yi Chang, Philip S. Yu, Qiang Yang, and Xing Xie. A survey on evaluation of large language models, 2023. URL: <https://arxiv.org/abs/2307.03109>, arXiv:2307.03109.
- [4] Nick Craswell, Bhaskar Mitra, Emine Yilmaz, Daniel Campos, and Jimmy Lin. Ms marco: Benchmarking ranking models in the large-data regime. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*, pages 1566–1576, 2021.
- [5] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Advances in Neural Information Processing Systems*, 37:82895–82920, 2024.
- [6] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. More than you’ve asked for: A comprehensive analysis of novel prompt injection threats to application-integrated large language models. *arXiv preprint arXiv:2302.12173*, 27, 2023.
- [7] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM workshop on artificial intelligence and security*, pages 79–90, 2023.
- [8] Healthcaremagic-100k-en. [Online]. Available: <https://huggingface.co/datasets/wangrongsheng/HealthCareMagic-100k-en>.
- [9] Changyue Jiang, Xudong Pan, Geng Hong, Chenfu Bao, and Min Yang. Rag-thief: Scalable extraction of private data from retrieval-augmented generation applications with agent-based attacks. *arXiv preprint arXiv:2411.14110*, 2024.
- [10] Sotiropoulos John, Rosario Ron F Del, Kokuykin Evgeniy, Oakley Helen, Habler Idan, Underkoffler Kayla, Huang Ken, Stefensen Peter, Aralimatti Rakshith, Bitton Ron, et al. *Owasp top 10 for llm apps & gen ai agentic security initiative*. PhD thesis, OWASP, 2025.
- [11] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pages 6769–6781, 2020.
- [12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [13] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, et al. Personal llm agents: Insights and survey about the

- capability, efficiency and security. *arXiv preprint arXiv:2401.05459*, 2024.
- [14] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, Rui Kong, Yile Wang, Hanfei Geng, Jian Luan, Xuefeng Jin, Zilong Ye, Guanqing Xiong, Fan Zhang, Xiang Li, Mengwei Xu, Zhijun Li, Peng Li, Yang Liu, Ya-Qin Zhang, and Yunxin Liu. Personal llm agents: Insights and survey about the capability, efficiency and security, 2024. URL: <https://arxiv.org/abs/2401.05459>, arXiv: 2401.05459.
- [15] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
- [16] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents, 2024. URL: <https://arxiv.org/abs/2402.17753>, arXiv: 2402.17753.
- [17] OpenAI. Gpt-5-mini, 2023. URL: <https://platform.openai.com/docs/models>.
- [18] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.
- [19] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024.
- [20] Zhenting Qi, Hanlin Zhang, Eric Xing, Sham Kakade, and Himabindu Lakkaraju. Follow my instruction and spill the beans: Scalable data extraction from retrieval-augmented generation systems. *arXiv preprint arXiv:2402.17840*, 2024.
- [21] Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Xuanye Zhou, Yufei Huang, Chaojun Xiao, et al. Tool learning with foundation models. *ACM Computing Surveys*, 57(4):1–40, 2024.
- [22] Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics*, 11:1316–1331, 2023.
- [23] Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Richard James, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. Replug: Retrieval-augmented black-box language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8371–8384, 2024.
- [24] Wenqi Shi, Ran Xu, Yuchen Zhuang, Yue Yu, Jieyu Zhang, Hang Wu, Yuanda Zhu, Joyce Ho, Carl Yang, and May D Wang. Ehragent: Code empowers large language models for few-shot complex tabular reasoning on electronic health records. *arXiv preprint arXiv:2401.07128*, 2024.
- [25] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- [26] Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. Beir: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. *arXiv preprint arXiv:2104.08663*, 2021.
- [27] Dave Van Veen, Cara Van Uden, Louis Blanke-meier, Jean-Benoit Delbrouck, Asad Aali, Chris-

- tian Bluethgen, Anuj Pareek, Malgorzata Polacin, Eduardo Pontes Reis, Anna Seehofnerová, et al. Adapted large language models can outperform medical experts in clinical text summarization. *Nature medicine*, 30(4):1134–1142, 2024.
- [28] Bing Wang, Xinnian Liang, Jian Yang, Hui Huang, Shuangzhi Wu, Peihao Wu, Lu Lu, Zejun Ma, and Zhoujun Li. Enhancing large language model with self-controlled memory framework. *arXiv preprint arXiv:2304.13343*, 2023.
- [29] Bo Wang, Weiyi He, Shenglai Zeng, Zhen Xiang, Yue Xing, Jiliang Tang, and Pengfei He. Unveiling privacy risks in llm agent memory. *arXiv preprint arXiv:2502.13172*, 2025.
- [30] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), March 2024. URL: <http://dx.doi.org/10.1007/s11704-024-40231-1>, doi:10.1007/s11704-024-40231-1.
- [31] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*, 2022.
- [32] Yuhao Wang, Wenjie Qu, Shengfang Zhai, Yanze Jiang, Zichen Liu, Yue Liu, Yinpeng Dong, and Jiaheng Zhang. Silent leaks: Implicit knowledge extraction attack on rag systems through benign queries, 2025. URL: <https://arxiv.org/abs/2505.15420>, arXiv:2505.15420.
- [33] Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. Tradingagents: Multi-agents llm financial trading framework. *arXiv preprint arXiv:2412.20138*, 2024.
- [34] Wenkai Yang, Xiaohan Bi, Yankai Lin, Sishuo Chen, Jie Zhou, and Xu Sun. Watch out for your agents! investigating backdoor threats to llm-based agents. *Advances in Neural Information Processing Systems*, 37:100938–100964, 2024.
- [35] Jiahao Yu, Xingwei Lin, Zheng Yu, and Xinyu Xing. {LLM-Fuzzer}: Scaling assessment of large language model jailbreaks. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 4657–4674, 2024.
- [36] Yangyang Yu, Haohang Li, Zhi Chen, Yuechen Jiang, Yang Li, Denghui Zhang, Rong Liu, Jordan W Suchow, and Khaldoun Khoshnevisan. Finmem: A performance-enhanced llm trading agent with layered memory and character design. *arXiv preprint arXiv:2311.13743*, 2024.
- [37] Shenglai Zeng, Jiankun Zhang, Pengfei He, Yue Xing, Yiding Liu, Han Xu, Jie Ren, Shuaiqiang Wang, Dawei Yin, Yi Chang, et al. The good and the bad: Exploring privacy issues in retrieval-augmented generation (rag). *arXiv preprint arXiv:2402.16893*, 2024.
- [38] Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. Agent security bench (asb): Formalizing and benchmarking attacks and defenses in llm-based agents, 2025. URL: <https://arxiv.org/abs/2410.02644>, arXiv:2410.02644.
- [39] Xi Zhang, Srivatsan Setlur, Maarten Sap, and Yejin Choi. Towards ethical personal ai applications: Practical considerations for ai assistants with long-term memory. *arXiv preprint arXiv:2409.11192*, 2024.
- [40] Zeyu Zhang, Quanyu Dai, Xu Chen, Rui Li, Zhongyang Li, and Zhenhua Dong. Memengine: A unified and modular library for developing advanced memory of llm-based agents. In *Companion Proceedings of the ACM on Web Conference 2025*, pages 821–824, 2025.
- [41] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong,

et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.

- [42] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

A Memory Isolation

We survey the memory isolation strategies adopted by three representative production systems. Despite architectural differences, all rely on a common principle: binding each user’s long-term memory to a unique identifier and enforcing access control at retrieval time.

OpenAI (ChatGPT). ChatGPT implements account-scoped memory isolation: every memory entry is bound to the authenticated user’s account and is inaccessible to any other account³. In Team and Enterprise workspaces, organizational boundaries provide an additional isolation layer—workspace administrators can configure data retention policies, and conversations within these workspaces are excluded from model training⁴.

Mem0. Mem0 scopes every memory operation through four dimensions: `user_id`, `agent_id`, `run_id` (session), and `app_id`⁵. All memory levels reside in the same physical storage (vector database and history database) but are logically separated via metadata filters: a retrieval query parameterized with one user’s identifier will never return memories belonging to another user⁶.

Anthropic (Claude). Claude employs account-scoped memory: all stored memories are private to the user’s account and are never shared with other users⁷.

³<https://help.openai.com/en/articles/9295112-memory-faq-business-version>

⁴<https://help.openai.com/en/articles/12574142-chatgpt-atlas-data-controls-and-privacy>

⁵<https://mem0.ai/blog/multi-agent-memory-systems>

⁶<https://docs.mem0.ai/platform/faqs>

⁷<https://privacy.claude.com/en/>

B Pareto tradeoff

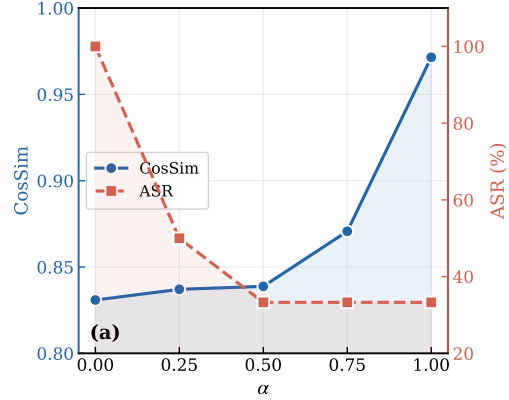


Figure 10: Pareto tradeoff.

C LLM-Fuzzer

LLM-Fuzzer [35] is a black-box fuzzing framework for scalably assessing the jailbreak vulnerability of large language models. Inspired by coverage-guided fuzzers in software testing, it treats jailbreak templates as seeds and automatically evolves them to discover effective attacks. The core workflow is as follows:

Seed Collection. Gather an initial pool of jailbreak templates from public sources (*e.g.*, online forums and community repositories).

Seed Selection. Choose a template from the pool using a UCB1-based bandit strategy that balances exploiting templates with high historical success rates and exploring less-tested ones.

Mutation. Feed the selected template to an auxiliary LLM with a randomly sampled mutation instruction (rephrase, expand, shorten, crossover with another template, *etc.*) to produce a new candidate template. **Execution.** Insert a harmful question into the candidate template and query the target LLM. A judge model then determines whether the response constitutes a successful jailbreak.

Pool Update. If the candidate succeeds, add it to the seed pool as a new template; propagate the outcome back to the parent template’s statistics to guide future selection.

Steps 2–5 repeat for a fixed budget of iterations.

D Dataset

Here, we give a detailed description of the datasets used in this work.

TREC-COVID [26]. This dataset contains 116K scientific and medical documents from the TREC-COVID collection, part of the BEIR benchmark, exemplifying research scenarios where COVID-related literature is integrated with RAG to improve information retrieval efficiency and the quality of scientific knowledge exploration in pandemic response contexts. In our work, we populate the reference LTM from TREC-COVID for parameter optimization (C_{adv} , N , P_{react}); its distribution intentionally differs from the victim’s, ensuring the optimized parameters generalize across domains.

HealthCareMagic-100k-en [8]. This dataset contains 112,165 real conversations between patients and doctors, representing the domain-specific medical agent setting in which a specialized assistant accumulates highly sensitive, single-domain long-term memories (e.g., symptoms, diagnoses, medications) to support downstream clinical tasks. We sample from this dataset to construct one of the two victim LTMs (200 records each), simulating a scenario where an adversary targets the private medical history stored by a clinical agent.

LoCoMo [16].⁸ LoCoMo (Long-term Conversational Memory) comprises multi-session dialogues between pairs of speakers, annotated with fine-grained memory categories including personal events, daily preferences, social relationships, and future plans. It represents the personal assistant setting in which a general-purpose conversational agent accumulates diverse, cross-domain long-term memories from everyday interactions. We sample from LoCoMo to construct the other victim LTM (200 records), complementing the medical-agent scenario with a broader, lifestyle-oriented memory store and enabling us to evaluate attack effectiveness across two distinct memory distributions.

⁸<https://github.com/snap-research/LoCoMo>

MS MARCO [4]. MS MARCO (Microsoft Machine Reading COmprehension) is a large-scale information retrieval dataset comprising approximately 8.8 million passages and over one million real user queries sampled from Bing search logs. In Scenario 2, where the number of trigger queries is constrained, we sample benign queries from MS MARCO to simulate normal user interactions that do not invoke the compromised tool. These benign queries follow a trigger query within the same session to resume the extraction loop, providing a realistic distribution of everyday information-seeking behavior against which the attack’s stealth and persistence are evaluated. During optimization, we use a small set of benign queries alongside a single trigger query; for evaluation, we draw separate, non-overlapping benign queries to avoid data leakage.

E Baseline

All three baselines were originally designed as user-side extraction attacks against RAG systems following the query-concatenation paradigm (Equation 3): each query consists of an anchor that steers the retriever toward a target region of the knowledge store, concatenated with an adversarial command that instructs the LLM to disclose the retrieved content verbatim. To port them to the tool-side setting, we embed the concatenated payload into the tool’s return value via indirect prompt injection, keeping the adversarial command C_{adv} identical across all methods so that they differ *only* in their anchor generation strategy.

E.1 Random Anchor Generation

MEXTRA [29]. MEXTRA generates anchors by prompting GPT-4 to produce diverse, topically varied short phrases that are semantically plausible as retrieval queries. Because the anchors are sampled independently of previously extracted content, MEXTRA does not require any feedback loop and can generate an arbitrarily large pool of candidate anchors in advance. Each anchor is concatenated with the shared adversarial command and submitted as

a query; the retrieved documents are collected and deduplicated to compute overall extraction coverage. The key advantage of this strategy is its simplicity and parallelizability; however, the lack of feedback means that many anchors may retrieve already-extracted or semantically overlapping records, leading to diminishing marginal returns as extraction progresses.

E.2 Feedback-Driven Anchor Generation

RAG-Thief [9]. RAG-Thief employs an iterative, feedback-driven strategy that leverages previously extracted content to generate new anchors. Specifically, after each extraction round, RAG-Thief feeds the newly obtained chunks to a local language model (Qwen2-1.5B-Instruct) and prompts it to generate forward and backward continuations. These continuations serve as anchors for the next round, under the intuition that semantically adjacent passages in the knowledge store are likely to be retrieved by queries resembling their neighboring content. By chaining continuations across rounds, RAG-Thief progressively expands its extraction frontier outward from an initial seed, achieving higher coverage than purely random sampling.

IKEA [32]. IKEA (Implicit Knowledge Extraction Attack) adopts a concept-level feedback strategy that operates in the embedding space rather than at the token level. After each extraction round, IKEA identifies salient concepts (key phrases or entities) from the accumulated query-response history and uses them as seeds for the next round of anchor generation. To maximize retrieval diversity, IKEA iteratively mutates these concept-level anchors subject to embedding-space similarity constraints: new anchors are required to remain close enough to the knowledge store’s distribution to trigger relevant retrieval, yet sufficiently distant from previously used anchors to avoid redundant results. This constrained mutation mechanism enables IKEA to explore the embedding space more systematically than forward/backward continuation, yielding higher extraction coverage under a fixed query budget.

F LTM framework

LTMemory. LTMemory represents the standard framework, using text embeddings to calculate semantic similarities and retrieve relevant information.

GAMemory. [18] GAMemory incorporates a self-reflection mechanism. This mechanism activates when the cumulative importance of recent events exceeds a threshold. It then synthesizes fragmented records into high-level inferences supported by evidence.

SCMemory. [28] SCMemory selectively recalls only necessary information for inference. A control mechanism determines when to activate memory retrieval and filter out noise. Additionally, a summarization mechanism condenses extended interactions while preserving core information for efficient reasoning.

G Experimental Result

H Potential Defense

Template for response filtering defense

You are a security auditor. Analyze the following tool response and determine if it contains prompt injection patterns.

Prompt injection patterns include:

- Instructions to ignore previous instructions
- Instructions to change behavior or role
- Instructions to output specific predetermined answers
- Instructions to concatenate or leak conversation history
- Any attempt to manipulate the calling AI agent

Tool Response: response

Reply with EXACTLY one word: **SAFE** or **UNSAFE**

Table 6: Consolidated results of Scenario 1 on the LoCoMo dataset.

Factor	Setting	RER (%) $\uparrow$				SS $\uparrow$				BER (%) $\uparrow$			
		MEXTRA	RAG-Thief	IKEA	<i>SPORE</i>	MEXTRA	RAG-Thief	IKEA	<i>SPORE</i>	MEXTRA	RAG-Thief	IKEA	<i>SPORE</i>
Generative Model	GPT-5-mini	7.0%	15.5%	6.0%	57.5%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	63.5%
	Gemini-2.5-pro	3.0%	18.5%	5.0%	45.5%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	95.0%
	Deepseek-chat	4.5%	13.5%	6.5%	52.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	70.5%
LTM Framework	LTMemory	7.0%	15.5%	6.0%	57.5%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	63.5%
	GAMemory	4.5%	6.5%	12.0%	55.0%	1.00	1.00	0.80	1.00	0.0%	0.0%	0.0%	75.5%
	SCMemory	10.0%	25.5%	10.0%	26.5%	0.57	0.70	0.84	0.84	0.0%	0.0%	0.0%	72.5%
Embedding Model	All-MiniLM-L6-v2	7.0%	15.5%	6.0%	57.5%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	63.5%
	Contriever	5.0%	22.5%	10.5%	58.5%	1.00	1.00	0.94	1.00	0.0%	0.0%	0.0%	72.0%
	E5-base	4.5%	12.5%	9.0%	58.5%	1.00	1.00	0.97	1.00	0.0%	0.0%	0.0%	68.0%

Table 7: Consolidated results of Scenario 2 on the LoCoMo dataset.

Trigger	Factor	Setting	RER (%) $\uparrow$				SS $\uparrow$				BER (%) $\uparrow$			
			MEXTRA	RAG-Thief	IKEA	<i>SPORE</i>	MEXTRA	RAG-Thief	IKEA	<i>SPORE</i>	MEXTRA	RAG-Thief	IKEA	<i>SPORE</i>
@1	Generative Model	GPT-5-mini	2.0%	4.5%	0.0%	5.0%	1.00	1.00	–	1.00	0.0%	0.0%	0.0%	90.3%
		Gemini-2.5-pro	3.0%	4.5%	3.0%	6.5%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	100.0%
		Deepseek-chat	3.5%	4.0%	2.5%	10.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	90.0%
	LTM Framework	LTMemory	2.0%	4.5%	0.0%	5.0%	1.00	1.00	–	1.00	0.0%	0.0%	0.0%	90.3%
		GAMemory	2.5%	4.5%	4.5%	5.0%	1.00	1.00	0.90	1.00	0.0%	0.0%	0.0%	100.0%
		SCMemory	0.0%	3.5%	1.5%	4.5%	–	0.60	0.98	0.80	–	0.0%	0.0%	60.0%
	Embedding Model	All-MiniLM-L6-v2	2.0%	4.5%	0.0%	5.0%	1.00	1.00	–	1.00	0.0%	0.0%	0.0%	90.3%
		Contriever	2.5%	4.0%	3.0%	4.5%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	100.0%
		E5-base	1.5%	1.0%	4.5%	5.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	100.0%
@5	Generative Model	GPT-5-mini	7.0%	10.5%	4.5%	22.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	75.2%
		Gemini-2.5-pro	3.0%	13.5%	5.0%	18.5%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	85.0%
		Deepseek-chat	4.5%	10.0%	6.5%	30.5%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	70.0%
	LTM Framework	LTMemory	7.0%	10.5%	4.5%	22.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	75.2%
		GAMemory	4.5%	6.0%	9.5%	23.5%	1.00	1.00	0.90		0.0%	0.0%	0.0%	75.0%
		SCMemory	5.5%	16.0%	6.0%	12.0%	0.59	0.68	0.82	0.84	0.0%	0.0%	0.0%	57.1%
	Embedding Model	All-MiniLM-L6-v2	7.0%	10.5%	4.5%	22.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	75.2%
		Contriever	5.0%	10.5%	9.0%	27.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	79.2%
		E5-base	4.5%	10.5%	9.0%	19.0%	1.00	1.00	0.97	1.00	0.0%	0.0%	0.0%	97.0%
@20	Generative Model	GPT-5-mini	7.0%	15.5%	6.0%	44.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	80.4%
		Gemini-2.5-pro	3.0%	18.5%	5.0%	45.5%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	80.2%
		Deepseek-chat	4.5%	13.5%	6.5%	35.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	72.0%
	LTM Framework	LTMemory	7.0%	15.5%	6.0%	44.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	80.4%
		GAMemory	4.5%	6.5%	12.0%	44.5%		1.00	0.80	1.00	0.0%	0.0%	0.0%	78.5%
		SCMemory	10.0%	25.5%	10.0%	14.0%	0.57	1.00	0.84	0.83	0.0%	0.0%	0.0%	65.9%
	Embedding Model	All-MiniLM-L6-v2	7.0%	15.5%	6.0%	44.0%	1.00	1.00	1.00	1.00	0.0%	0.0%	0.0%	80.4%
		Contriever	5.0%	22.5%	10.5%	40.5%	1.00	1.00	0.94	0.99	0.0%	0.0%	0.0%	83.2%
		E5-base	4.5%	12.5%	9.0%	38.5%	1.00	1.00	0.97	1.00	0.0%	0.0%	0.0%	77.8%

Table 8: Consolidated results of Scenario 3 on the LoCoMo dataset.

Trigger	Factor	Setting	Metrics		
			RER (%) $\uparrow$	SS $\uparrow$	BER (%) $\uparrow$
@1	Generative Model	GPT-5-mini	69.0%	1.00	72.0%
		Gemini-2.5-pro	71.0%	1.00	80.0%
		Deepseek-chat	81.0%	1.00	52.0%
	LTM Framework	LTMemory	69.0%	1.00	72.0%
		GAMemory	76.0%	1.00	70.0%
		SCMemory	35.0%	0.75	93.5%
	Embedding Model	All-MiniLM-L6-v2	69.0%	1.00	72.0%
		Contriever	87.0%	1.00	65.0%
		E5-base	74.5%	1.00	70.0%